

Homework 05

Question 1

Solution:

```
f <- function(X) 0.5*X[,1] + sin(X[,2]) - 0.3*X[,3]^2

x0 <- c(0.5, 0.7, 1.0)
fx0 <- 0.5*x0[1] + sin(x0[2]) - 0.3*x0[3]^2
cat("f(x0) =", fx0, "\n")
```

f(x0) = 0.5942177

```
ks <- seq(1, 29, by = 2) # 1, 3, ..., 29

knn_predict_one <- function(Xtrain, ytrain, x0, ks) {
  d2 <- rowSums(sweep(Xtrain, 2, x0, "-")^2)
  ord <- order(d2, method = "radix") # stable -> ties by row order
  y_sorted <- ytrain[ord]
  sapply(ks, function(k) mean(y_sorted[1:k]))
}
```

Solution Q1a.

```
set.seed(432)
n <- 400
X <- matrix(rnorm(n * 3), nrow = n)
y <- f(X) + rnorm(n, sd = 0.1)
preds_a <- knn_predict_one(X, y, x0, ks)
data.frame(k = ks, prediction = round(preds_a, 4))
```

	k	prediction
1	1	0.7520
2	3	0.7520
3	5	0.5437
4	7	0.4586
5	9	0.4823
6	11	0.4087
7	13	0.4541
8	15	0.5077
9	17	0.5448
10	19	0.5171
11	21	0.4975
12	23	0.5478
13	25	0.5455
14	27	0.4762
15	29	0.4563

Solution Q1b.

```
set.seed(4320)
R <- 200
pred_matrix <- matrix(0, nrow = R, ncol = length(ks))
for (r in 1:R) {
  Xr <- matrix(rnorm(n * 3), nrow = n)
  yr <- f(Xr) + rnorm(n, sd = 0.1)
  pred_matrix[r, ] <- knn_predict_one(Xr, yr, x0, ks)
}

bias <- colMeans(pred_matrix) - fx0
var_hat <- apply(pred_matrix, 2, function(col) mean((col - mean(col))^2)) # divisor 200
mse <- colMeans((pred_matrix - fx0)^2)
sqbias <- bias^2

cat("max |MSE - (sqBias + Var)| =", max(abs(mse - (sqbias + var_hat))), "\n")
```

```
max |MSE - (sqBias + Var)| = 4.857226e-17
```

The identity $\text{MSE} = \text{squared bias} + \text{variance}$ holds numerically to within floating-point error, confirming the decomposition.

```

plot(ks, sqbias, type = "b", pch = 16, col = "#0072B2",
     ylim = range(c(sqbias, var_hat, mse)),
     xlab = "Number of neighbors (k)", ylab = "Value",
     main = "KNN bias^2, variance, and MSE at x0 vs k")
lines(ks, var_hat, type = "b", pch = 15, col = "#D55E00")
lines(ks, mse, type = "b", pch = 17, col = "#009E73")
legend("topright", legend = c("Squared bias", "Variance", "MSE"),
     col = c("#0072B2", "#D55E00", "#009E73"), pch = c(16,15,17), bty = "n")

```

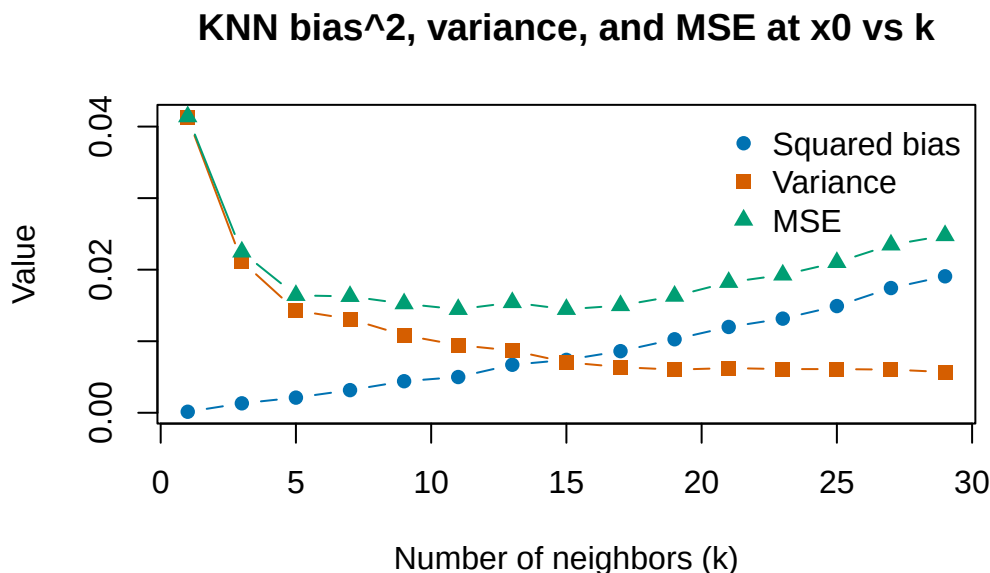


Figure 1: Squared bias, variance, and MSE of the KNN estimate at x_0 , vs k

Pattern. Squared bias climbs pretty steadily as k grows. That's not surprising: bringing in more neighbors means averaging over points that sit farther from x_0 , and the true function is curving locally in a way a wider average can't track. Variance goes the other way, and for a good reason: once you're averaging k roughly independent noisy points, the sampling variance falls off close to $1/k$. Put the two together and MSE traces a U: variance runs the show while k is small, bias takes over once k gets large.

Comparison with ridge/lasso. λ ends up playing the same role here that k does. $\lambda = 0$ is basically $k = 1$: no bias, but the variance is bad. Turning λ up shrinks that variance at the cost of bias, and you get the same U-shaped MSE curve out the other end. What's actually different is how the tradeoff gets made. Ridge and lasso get there by shrinking or

zeroing coefficients inside a linear model that never changes shape. KNN doesn't touch any coefficients at all — it just changes how many neighbors go into the average. So the curve looks the same, but one method is shrinking a fixed parametric fit and the other never had a global functional form to shrink.

Question 2

Solution Q2:

Setup-

```
library(glmnet)

p <- 30
n_train <- 400
n_test <- 1000
sigma <- 0.1
lambdas <- sort(10^seq(-4, 0, length.out = 41), decreasing = TRUE)

knn_predict_many <- function(Xtrain, ytrain, Xtest, k) {
  apply(Xtest, 1, function(x0row) {
    d2 <- rowSums(sweep(Xtrain, 2, x0row, "-")^2)
    ord <- order(d2, method = "radix")
    mean(ytrain[ord[1:k]])
  })
}

run_setting <- function(seed, corr) {
  set.seed(seed)
  if (!corr) {
    Xtr <- matrix(rnorm(n_train * p), nrow = n_train)
    Xte <- matrix(rnorm(n_test * p), nrow = n_test)
  } else {
    Sigma <- matrix(0.8, p, p); diag(Sigma) <- 1
    Lchol <- chol(Sigma)
    Xtr <- matrix(rnorm(n_train * p), nrow = n_train) %*% Lchol
    Xte <- matrix(rnorm(n_test * p), nrow = n_test) %*% Lchol
  }
  ytr <- f(Xtr) + rnorm(n_train, sd = sigma)
  yte <- f(Xte) + rnorm(n_test, sd = sigma)

  knn_pred <- knn_predict_many(Xtr, ytr, Xte, k = 5)
```

```

knn_mse <- mean((knn_pred - yte)^2)

Xtr_mean <- colMeans(Xtr)
Xtr_sd <- apply(Xtr, 2, sd) * sqrt((n_train - 1) / n_train)
ytr_mean <- mean(ytr)
Xtr_c <- scale(Xtr, center = Xtr_mean, scale = Xtr_sd)
ytr_c <- ytr - ytr_mean

cvfit <- cv.glmnet(Xtr_c, ytr_c, alpha = 1, lambda = lambdas, nfolds = 10,
                  standardize = FALSE, intercept = FALSE)
best_lambda <- cvfit$lambda.min
coefs <- as.numeric(coef(cvfit, s = "lambda.min"))[-1] # drop intercept row (0)
nnz <- sum(abs(coefs) > 1e-10)

Xte_c <- scale(Xte, center = Xtr_mean, scale = Xtr_sd)
lasso_pred <- as.numeric(predict(cvfit, newx = Xte_c, s = "lambda.min")) + ytr_mean
lasso_mse <- mean((lasso_pred - yte)^2)

c(knn_mse = knn_mse, lasso_mse = lasso_mse, lambda = best_lambda, nnz = nnz)
}

res1 <- run_setting(seed = 100, corr = FALSE)
res2 <- run_setting(seed = 200, corr = TRUE)

```

Q2a Solution:

```

tab <- rbind(
  "1: independent" = res1,
  "2: correlated (rho=0.8)" = res2
)
knitr::kable(round(tab, 5))

```

	knn_mse	lasso_mse	lambda	nnz
1: independent	0.53652	0.24780	0.05012	2
2: correlated (rho=0.8)	0.16610	0.25931	0.03162	3

Seeds used: **100** (Setting 1), **200** (Setting 2).

Solution Q2b.

With 30 independent covariates, 27 of them are pure noise, and Euclidean distance can't tell the difference. That means 5NN's neighborhoods end up close to random with respect to (x_1, x_2, x_3) , and its test MSE is much worse than lasso's here. Correlation fixes a good chunk of that problem: at $\rho = 0.8$ all 30 variables move together, so the data crowd toward a lower-dimensional subspace, and the noise directions stop dominating distance the way they did before. Neighbors in X-space start actually being closer in (x_1, x_2, x_3) -space too, which is why 5NN's MSE drops under correlation. Lasso isn't playing this game at all. It never relies on proximity — it just selects a small active set through the l1 penalty and fits a coefficient to each one, so its performance comes down to whether a linear combination of a few covariates can approximate f . Correlation only enters indirectly, by shifting which variables get picked.

Solution Q2c.

Lasso genuinely can't represent $\sin(x_2)$ or $-0.3x_3^2$, so it's carrying model bias no matter what λ or correlation structure you give it. KNN doesn't have that ceiling — it can pick up the curvature, at the cost of its own bias-variance tradeoff in k . That's a real asymmetry, and it means part of what's going on in this comparison has nothing to do with neighborhoods at all; it's just lasso being the wrong model class. Given that, I'd push back on generalizing "correlation helps KNN" from this one result. It helped here specifically because the correlated directions reinforced the actual signal — if the correlated block had been mostly noise, the same mechanism could easily have hurt instead. The direction correlation pushes things depends entirely on whether it's amplifying signal or amplifying junk, and this setup happened to amplify signal.

Question 3

Solution Q3:

```
p <- 100; n_train <- 200; n_test <- 200
ks3 <- seq(2, 82, by = 8) # 2, 10, ..., 82

one_dataset <- function(seed, m) {
  set.seed(seed)
  A <- matrix(runif(p * m, -2, 2), nrow = p)
  Ztr <- matrix(rnorm(n_train * m), nrow = n_train)
  Zte <- matrix(rnorm(n_test * m), nrow = n_test)
  Xtr <- Ztr %*% t(A) + matrix(rnorm(n_train * p), nrow = n_train)
  Xte <- Zte %*% t(A) + matrix(rnorm(n_test * p), nrow = n_test)
```

```

ytr <- Ztr[,1] + Ztr[,2] + rnorm(n_train)
yte <- Zte[,1] + Zte[,2] + rnorm(n_test)
list(Xtr = Xtr, ytr = ytr, Xte = Xte, yte = yte)
}

mse_curve <- function(d, ks) {
  sapply(ks, function(k) {
    pred <- knn_predict_many(d$Xtr, d$ytr, d$Xte, k)
    mean((pred - d$yte)^2)
  })
}

```

Q3a Solution:

Seeds used: **321** ($m = 3$) and **654** ($m = 30$).

```

d3 <- one_dataset(321, 3)
d30 <- one_dataset(654, 30)
mse_m3 <- mse_curve(d3, ks3)
mse_m30 <- mse_curve(d30, ks3)
data.frame(k = ks3, m3 = round(mse_m3,3), m30 = round(mse_m30,3))

```

	k	m3	m30
1	2	1.545	3.464
2	10	1.183	2.745
3	18	1.136	2.679
4	26	1.156	2.652
5	34	1.192	2.706
6	42	1.251	2.727
7	50	1.277	2.770
8	58	1.310	2.812
9	66	1.350	2.826
10	74	1.389	2.876
11	82	1.427	2.906

Q3b Solution:

Seed used: **999** for the overall repetition loop (a fresh seed is set per repetition, per m , before drawing $\mathbf{A}$, $\mathbf{Z}$, η , and ϵ).

```

set.seed(999)
Rrep <- 50
mse3 <- matrix(0, Rrep, length(ks3))
mse30 <- matrix(0, Rrep, length(ks3))
for (r in 1:Rrep) {
  mse3[r, ] <- mse_curve(one_dataset(1000 + r, 3), ks3)
  mse30[r, ] <- mse_curve(one_dataset(2000 + r, 30), ks3)
}
avg3 <- colMeans(mse3)
avg30 <- colMeans(mse30)

plot(ks3, avg3, type = "b", pch = 16, col = "#0072B2",
     ylim = range(c(avg3, avg30)),
     xlab = "Number of neighbors (k)", ylab = "Average test MSE",
     main = "Average test MSE vs k")
lines(ks3, avg30, type = "b", pch = 15, col = "#D55E00")
legend("topright", legend = c("m = 3", "m = 30"),
     col = c("#0072B2", "#D55E00"), pch = c(16, 15), bty = "n")

```

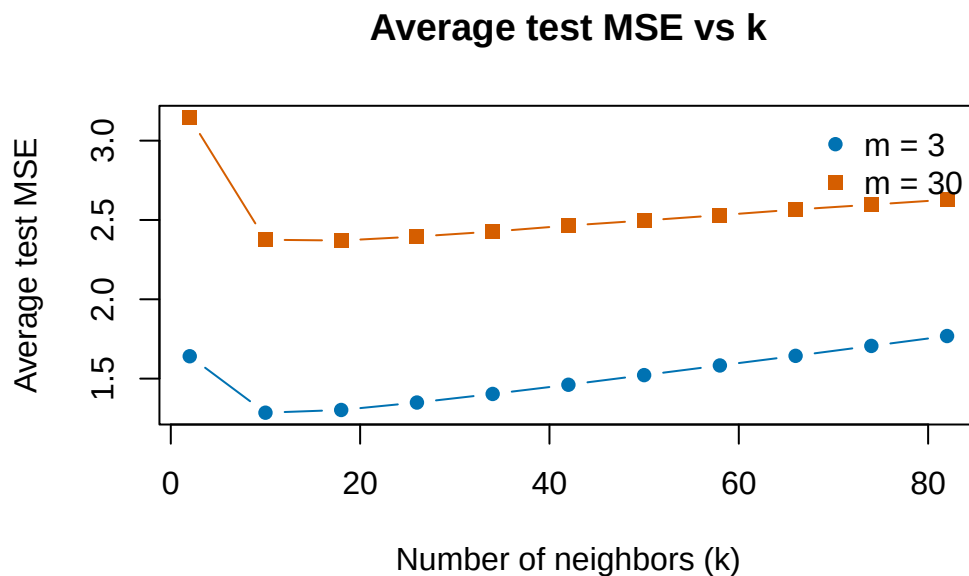


Figure 2: Average test MSE vs k, for $m = 3$ and $m = 30$ (50 repetitions)

Q3c Solution:

Both settings have $p = 100$ observed covariates, but that number is misleading on its own; what matters is how much of that space is actually carrying signal. When $m = 3$, $\mathbf{X}_i = \mathbf{A}\mathbf{Z}_i + \eta_i$ sits close to a 3-dimensional subspace, so Z_1 and Z_2 , the two directions the response actually depends on, make up a large share of the structured variation in $\mathbf{X}$. Neighbors in the observed space end up being real neighbors in (Z_1, Z_2) too. Bump m to 30 and the same total variance gets smeared across many more latent directions, so Z_1 and Z_2 now contribute a much smaller fraction of the distance calculation. Neighborhoods get noisier with respect to Y , and test MSE goes up accordingly. I think this one is fairly clear-cut, unlike the correlation question above: adding latent dimensions dilutes signal-bearing directions in Euclidean distance essentially by construction, and there's no scenario here where more m would help. The noise η_i is what keeps $\mathbf{X}_i$ off the exact m -dimensional subspace spanned by $\mathbf{A}$; remove it and $\mathbf{X}_i$ is a noiseless linear function of $\mathbf{Z}_i$, at which point the 100 observed covariates carry no more information than the m latent ones did. With η_i present, distance in the observed space mixes real latent-subspace signal with 100 coordinates of idiosyncratic noise, and that mixture gets worse the larger m is.

Question 4

```
train <- read.csv(gzfile("data/knn/zip-train.csv.gz"), header = FALSE)
test  <- read.csv(gzfile("data/knn/zip-test.csv.gz"),  header = FALSE)

train <- train[1:1000, ]
test  <- test[1:500, ]

ytr <- as.integer(train[, 1]); Xtr <- as.matrix(train[, -1])
yte <- as.integer(test[, 1]);  Xte <- as.matrix(test[, -1])
```

Q4a Solution:

```
knn5_classify <- function(Xtrain, ytrain, Xtest, metric = c("euclidean", "manhattan")) {
  metric <- match.arg(metric)
  n_test <- nrow(Xtest)
  preds <- integer(n_test)
  for (i in seq_len(n_test)) {
    diff <- sweep(Xtrain, 2, Xtest[i, ], "-")
    dist <- if (metric == "euclidean") sqrt(rowSums(diff^2)) else rowSums(abs(diff))
    ord <- order(dist, method = "radix")
```

```

    nn_labels <- ytrain[ord[1:5]]
    counts <- tabulate(nn_labels + 1, nbins = 10)
    preds[i] <- which.max(counts) - 1
  }
  preds
}

confusion_matrix_10 <- function(y_true, y_pred) {
  M <- matrix(0L, 10, 10, dimnames = list(pred = 0:9, true = 0:9))
  for (i in seq_along(y_true)) M[y_pred[i] + 1, y_true[i] + 1] <- M[y_pred[i] + 1, y_true[i] + 1] + 1
  M
}

```

Distances here are computed directly rather than through a built-in function, and both tie-breaking rules are handled explicitly since the assignment asks for that. `order(..., method = "radix")` is a stable sort, so when two training points are equidistant, the one with the smaller row number wins. `which.max()` returns the first maximum it finds, which is why voting ties resolve to the smallest digit label rather than something arbitrary.

Q4b Solution:

```

pred_eu <- knn5_classify(Xtr, ytr, Xte, "euclidean")
pred_mh <- knn5_classify(Xtr, ytr, Xte, "manhattan")

err_eu <- mean(pred_eu != yte)
err_mh <- mean(pred_mh != yte)

cm_eu <- confusion_matrix_10(yte, pred_eu)
cm_mh <- confusion_matrix_10(yte, pred_mh)

cat("Euclidean test error:", err_eu, "\n")

```

Euclidean test error: 0.122

```
print(cm_eu)
```

```

      true
pred  0  1  2  3  4  5  6  7  8  9
0 118  0  7  1  0  4  2  0  2  0

```

1	0	66	0	0	2	0	0	0	0	1
2	0	0	39	1	0	0	2	0	0	1
3	0	0	0	23	0	4	0	0	2	0
4	0	0	1	0	27	1	1	1	0	1
5	0	0	0	1	0	13	0	0	0	0
6	0	2	0	0	0	1	41	0	2	0
7	0	0	0	0	1	1	0	34	0	4
8	0	0	3	2	0	2	0	0	37	0
9	1	0	0	1	4	0	0	1	1	41

```
cat("\nManhattan test error:", err_mh, "\n")
```

Manhattan test error: 0.138

```
print(cm_mh)
```

	pred	0	1	2	3	4	5	6	7	8	9
true	0	118	0	6	1	0	5	3	0	4	0
1	0	67	2	0	2	0	1	0	0	0	1
2	0	0	35	0	0	0	2	0	0	0	0
3	0	0	1	24	0	4	0	0	0	2	0
4	0	0	1	1	27	1	0	1	0	1	1
5	0	0	0	1	0	11	0	0	0	0	0
6	0	1	0	0	0	2	40	0	2	0	0
7	0	0	2	1	1	0	0	34	0	5	0
8	0	0	2	0	0	3	0	0	34	0	0
9	1	0	1	1	4	0	0	1	2	41	0

Q4c Solution:

```
wrong <- which(pred_eu != yte)[1:3]
par(mfrow = c(1,3), mar = c(1,1,3,1))
for (i in wrong) {
  img <- matrix(as.numeric(Xte[i, ]), nrow = 16, ncol = 16)
  img <- t(img[nrow(img):1, ]) # orient like the digit image
  image(img, col = gray.colors(256), axes = FALSE,
        main = paste0("true=", yte[i], ", pred=", pred_eu[i]))
}
```

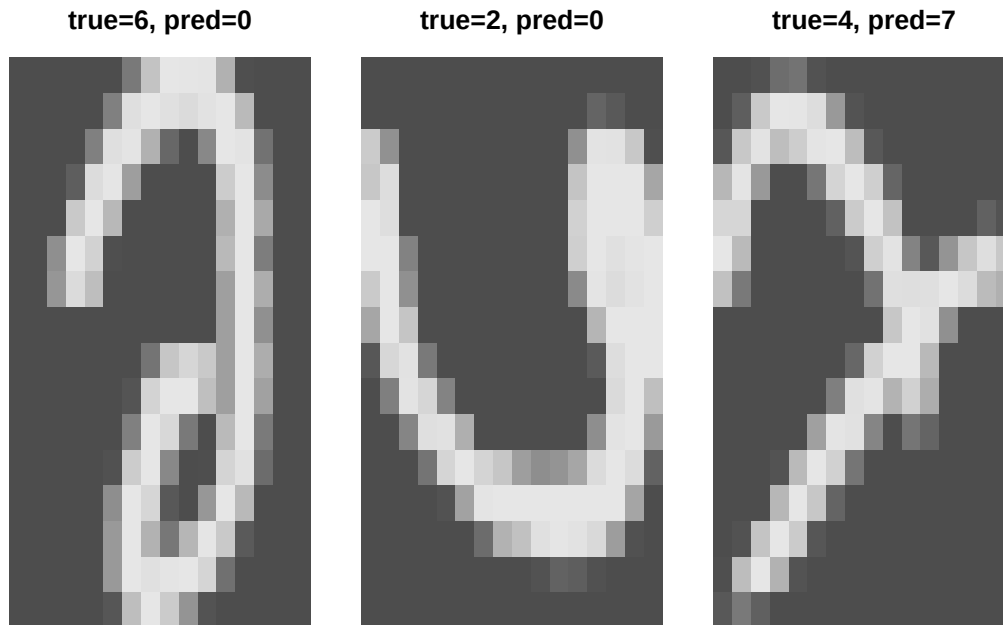


Figure 3: Three Euclidean-distance misclassifications

Looking at the three misclassified images, none of them feel like mistakes exactly. The 6 that got called a 0 is drawn with a closed, rounded loop instead of the usual open tail, so it genuinely looks like a 0. The 2 that got called a 0 has a top stroke that curls all the way around into a closed loop. The 4 read as a 7 has an open, rounded top that overlaps with what a 7's crossbar looks like. These are believable handwriting styles, not data errors.

```
wrong_eu <- which(pred_eu != yte)
wrong_mh <- which(pred_mh != yte)
common   <- intersect(wrong_eu, wrong_mh)
cat("Errors Euclidean:", length(wrong_eu),
    " Errors Manhattan:", length(wrong_mh),
    " Common:", length(common),
    " Only Euclidean:", length(setdiff(wrong_eu, wrong_mh)),
    " Only Manhattan:", length(setdiff(wrong_mh, wrong_eu)), "\n")
```

Errors Euclidean: 61 Errors Manhattan: 69 Common: 58 Only Euclidean: 3 Only Manhattan: 1

Question 5

(a)-(b) Skill and before/after comparison

The SKILL.md below only activates on explicit request and is scoped to formatting STAT 432 plots specifically. I applied it to the Question 1(b) plot: colorblind-safe palette, larger and more consistent fonts, light gridlines, tighter margins, a cleaner legend. The statistical content is unchanged. The figure is easier to read at a glance now, mostly because of the color contrast and the reclaimed margin space, though it's a modest change rather than a dramatic one.

(c) Final SKILL.md

```
---
name: stat432-plot-polish
description: Improves the formatting of a STAT 432 homework plot (R or Python) when the user
---

# STAT 432 Homework Plot Formatting

A short checklist for turning a working R/Python plot into a clean, submission-ready figure.
Apply only to the plot(s) named by the user; do not regenerate the underlying analysis.

## Rules

1. Margins: leave enough margin that no title, axis label, or tick label is clipped;
   avoid large unused whitespace on any side.
2. Titles: add a concise, descriptive title stating what is plotted (quantity vs. quantity);
   do not leave a plot untitled or reuse a generic default title.
3. Axes: label both axes with the quantity and, if applicable, units; use a sensible
   numeric range (don't let padding dominate the data); use gridlines only if they aid reading.
4. Labels/Legend: if more than one series/group is shown, add a legend or direct labels
   identifying each one; keep legend text short and non-overlapping with the data.
5. Colors: use a colorblind-friendly, distinguishable palette; keep it consistent across
   panels of the same submission; avoid default overly-saturated or clashing colors.
6. Sizes: use a consistent, readable font size for title/axis/legend text (title slightly
   larger than axis text); size the figure so it is legible at the width it will be viewed
   (e.g., ~6x4 in for a single plot).

## Output

Return the revised plotting code (R or Python) with these rules applied, changing only format
```